

SYSTEM ANALYSIS AND DESIGN

BIT 123 • Pre-Board Examination Preparation

Complete Answers to Model Sets 1–4

Based strictly on the questions and scope provided in the teacher's model examination paper.

Program	Bachelor of Information Technology (Hons)
Module	BIT 123 – System Analysis and Design
Exam Format	Section A: $10 \times 2 = 20$ Section B: $5 \times 7 = 35$ Section C: $3 \times 15 = 45$
Coverage	Model Sets 1, 2, 3 and 4

How to use this file

- Section A answers are concise enough to reproduce in a 2-mark answer while containing the key definition and distinction.
- Section B answers are organized into explanation points suitable for approximately 7 marks.
- Section C answers are deliberately detailed and include text-based diagrams/tables where the question asks for a diagram.
- In the real examination, draw diagrams neatly with boxes, arrows, crow's-foot/Chen notation, and labels. The text diagrams here are revision-ready representations.
- All questions in all four supplied model sets are answered; none of the supplied questions are intentionally skipped.

Source note: The question paper supplied contains four model sets for BIT 123, System Analysis and Design, Year 2/Semester 4, with Section A, B and C instructions and the case studies reproduced across the sets. □filecite□turn0file0□L19-L27□

MODEL SET 1 — COMPLETE ANSWERS

The supplied Set 1 specifies 10 very-short questions (2 marks each), six short questions with five to be attempted (7 marks each), and four long questions with the case study compulsory.

□filecite□turn0file0□L28-L55□

1. Define System Analysis and mention any one primary objective of performing analysis in SDLC. [2 Marks]

System Analysis is the systematic study of an existing or proposed information system to understand its processes, data, requirements, problems, and opportunities. Its primary objective is to identify and clearly document user/business requirements so that the proposed system solves the right problems and meets organizational needs.

2. State any two major characteristics of an Information System. [2 Marks]

1. An information system collects, processes, stores, and distributes information to support organizational operations and decision-making. 2. It consists of interrelated components such as people, procedures, hardware, software, data, and communication/network resources working toward a common objective.

3. Define Data Flow Diagram (DFD) and state its primary role in structured analysis. [2 Marks]

A Data Flow Diagram (DFD) is a graphical model showing how data enters, moves through, is processed by, and leaves a system, including external entities and data stores. Its primary role in structured analysis is to represent system processes and data movement clearly and logically.

4. Mention any two standard graphical symbols used in an Entity Relationship Diagram (ERD). [2 Marks]

Two standard ERD symbols are: (1) rectangle for an entity, and (2) oval/ellipse for an attribute. Relationships are commonly represented by diamonds in Chen notation.

5. What is a Data Dictionary? State two entries commonly cataloged within it. [2 Marks]

A Data Dictionary is a centralized repository containing definitions and specifications of data used by a system. Two common entries are data elements (for example, Student_ID and Student_Name) and data structures/data stores (for example, Student_Record or Student_File).

6. Define Decision Table and identify its four essential quadrant components. [2 Marks]

A Decision Table is a tabular technique for representing complex decision logic by listing conditions and corresponding actions. Its four components are: condition stub, action stub, condition entries, and action entries.

7. State any two distinct types of feasibility studies conducted during system planning. [2 Marks]

Any two are: Technical feasibility, which checks whether required technology, skills, infrastructure, and resources are available; and Economic feasibility, which checks whether expected benefits justify development and operating costs.

8. Define Logical Design and distinguish it briefly from Physical Design. [2 Marks]

Logical Design specifies what the system must contain and how components/data are logically organized, without committing to specific technology. Physical Design converts that logical specification into actual implementation details such as DBMS structures, hardware, software, network configuration, and physical files.

9. Define coupling and cohesion in the context of modular system design. [2 Marks]

Coupling is the degree of dependency or interconnection between modules; lower/looser coupling is generally desirable. Cohesion is the degree to which the elements within a module belong together and serve one focused purpose; higher cohesion is generally desirable.

10. Define system testing and state any two types of software maintenance. [2 Marks]

System testing evaluates the complete integrated system against specified requirements. Two types of maintenance are corrective maintenance (fixing faults) and adaptive maintenance (modifying software for changes in its environment).

SECTION B — SHORT ANSWERS

11. Explain the role of System Analysis and discuss the core technical, analytical, and interpersonal responsibilities of a System Analyst. [7 Marks]

System Analysis is the process of examining an organization's current system, identifying problems and requirements, and developing a clear logical description of the proposed solution. It connects business needs with technical design.

Core responsibilities of a System Analyst:

1. Technical responsibilities: understand system technologies, data, networks, software, interfaces, security, and modeling techniques; prepare models such as DFDs, ERDs and specifications.
2. Analytical responsibilities: investigate the existing system, identify problems and causes, analyze requirements, evaluate alternatives, assess feasibility, and determine how processes and data should be improved.
3. Requirements responsibility: gather, validate, prioritize, document, and manage functional and non-functional requirements.
4. Communication responsibility: communicate with users, managers, developers, and other stakeholders so that business expectations are correctly translated into technical requirements.
5. Interpersonal responsibility: conduct interviews and meetings, resolve conflicting requirements, negotiate practical alternatives, and build cooperation among stakeholders.
6. Documentation responsibility: maintain clear requirement specifications, models, data definitions, decisions, and traceability.
7. Overall role: act as a bridge between organizational users and the development team, reducing misunderstanding and ensuring that the proposed system is useful, feasible, and aligned with organizational goals.

12. Explain the concept of a Context Diagram (Level 0 DFD). Discuss the significance and rules of decomposing it into a Level 1 Data Flow Diagram. [7 Marks]

A Context Diagram, also called a Level 0 DFD, presents the entire information system as one high-level process. It shows the system boundary, external entities interacting with the system, and the major data flows entering and leaving the system. Internal processes and internal data stores are normally not shown at this level.

Significance:

1. Gives a simple overview of the system boundary and major interactions.
2. Identifies external entities and their relationship with the system.
3. Establishes the major input and output data flows.
4. Provides the starting point for detailed DFD decomposition.

Rules for Level 1 decomposition:

1. The single Level 0 process is broken into major internal processes.
2. Major inputs and outputs should be preserved through balancing; a data flow entering/leaving the parent process should have a corresponding flow at the child level.
3. Each process should have meaningful inputs and outputs.
4. Data stores may be introduced when internal data must be stored or retrieved.
5. External entities generally remain outside the system boundary.
6. Processes should be numbered consistently, for example 1.0, 2.0, 3.0.
7. Decomposition should stop at a level where each process is sufficiently simple to understand and specify.

13. Discuss the purpose of Data and Process Modeling tools. Explain the role and advantages of Computer-Aided Software Engineering (CASE) tools in system development. [7 Marks]

Data and process modeling tools provide structured ways to understand how information is represented and how business processes transform that information. Examples include DFDs for data movement, ERDs for data relationships, decision tables/trees for logic, and structure charts for modular design.

Purpose:

1. Reduce complexity by representing a system visually.
2. Identify inputs, outputs, processes, data stores, entities and relationships.
3. Improve communication between analysts, users and developers.
4. Detect missing, duplicated or inconsistent requirements.
5. Provide documentation for later design, implementation and maintenance.

CASE tools are Computer-Aided Software Engineering tools that support activities across the system development life cycle.

Advantages:

1. Diagram generation: quickly create and update DFDs, ERDs, UML diagrams and other models.
2. Repository management: store shared definitions of data, processes, models and project artifacts.
3. Consistency checking: help identify naming or modeling inconsistencies.
4. Code skeleton generation: some tools can generate basic program/database structures from models.
5. Traceability: connect requirements with analysis, design and implementation artifacts.
6. Productivity: automate repetitive documentation and modeling work.
7. Maintenance: centralized project information makes later modification and impact analysis easier.

14. Explain Decision Trees and Structured English. Compare their suitability with Decision Tables when modeling complex business logic. [7 Marks]

A Decision Tree is a graphical representation of decision logic. It starts with a condition and branches according to possible outcomes until an action/result is reached. It is useful when the sequence of decisions is important and the logic can be understood visually.

Structured English is a restricted form of natural language using programming-like constructs such as IF, ELSE, DO WHILE and THEN. It describes process logic in a precise but readable form.

Comparison:

- Decision Tree: best when decisions form a branching sequence and a visual path from condition to action is useful.
- Structured English: best when a process contains sequential steps, nested conditions, or repetition and the

analyst wants readable procedural logic.

- Decision Table: best when many combinations of conditions must be checked systematically. It reduces the risk of overlooking a combination and makes rules explicit in rows/columns.
- For highly complex business rules, Decision Tables are often useful for exhaustive condition combinations, while Decision Trees are easier to follow for sequential alternatives and Structured English is easier for detailed procedural actions.

Thus, the choice depends on the structure of the logic: branching → tree; procedural/nested logic → Structured English; many condition combinations → Decision Table.

15. Discuss the Feasibility Study process in depth. Explain the significance of Cost/Benefit Analysis (distinguishing direct vs. indirect costs and tangible vs. intangible benefits). [7 Marks]

A feasibility study determines whether a proposed system is practical and worth pursuing before major resources are committed.

Main process:

1. Define the proposed system, objectives, scope and major constraints.
2. Gather preliminary requirements and understand the existing environment.
3. Identify alternative solutions.
4. Assess technical feasibility: technology, infrastructure, skills and integration.
5. Assess operational feasibility: organizational fit, procedures and user acceptance.
6. Assess economic feasibility: costs, benefits, return and financial viability.
7. Assess schedule feasibility: whether the system can be delivered within the required time.
8. Prepare a feasibility report and recommend a viable alternative based on evidence.

Cost/Benefit Analysis compares expected system costs with expected benefits.

Direct costs are directly attributable to the project, such as developer salaries, purchased servers, software licenses and implementation expenses.

Indirect costs are supporting or consequential costs that are not easily assigned to one activity, such as general administration, temporary productivity loss, or broader support overhead.

Tangible benefits can be measured financially, such as reduced printing costs, fewer staff hours, lower error-processing costs, or increased sales.

Intangible benefits are difficult to express directly in money, such as improved customer satisfaction, better employee morale, stronger organizational image, and improved decision quality.

Cost/Benefit Analysis helps management compare alternatives and determine whether the expected value of the system justifies its investment.

16. Explain Structured System Design. Discuss modular design, functional strength, structure charts, and techniques for minimizing module coupling. [7 Marks]

Structured System Design converts analyzed requirements into a modular technical structure. The objective is to organize the system into manageable modules with clear responsibilities and controlled interfaces.

1. Modular design: divide a large system into smaller modules. Each module should perform a meaningful task and have a clearly defined interface.

2. Functional strength (cohesion): measures how closely related the responsibilities within one module are. Functional cohesion is desirable because a module focused on one task is easier to understand, test and maintain.
3. Structure charts: hierarchical diagrams showing modules, module calls, data/control relationships and the organization of a system.
4. Coupling: measures dependency between modules. Lower coupling makes changes less likely to propagate.

Techniques to minimize coupling:

- Pass only necessary data rather than large unrelated structures.
- Avoid unnecessary global data.
- Use simple, well-defined interfaces.
- Keep control information to a minimum.
- Avoid one module directly changing another module's internal data.
- Encapsulate internal details and communicate through defined interfaces.
- Use data coupling where modules exchange only the required data.

The overall aim is high cohesion within modules and loose coupling between modules, producing maintainable and testable software.

SECTION C — LONG ANSWERS

17. COMPULSORY CASE STUDY — ACADEMIC INSTITUTION AUTOMATION [15 Marks]

Scenario summary: The institution is replacing paper-based admissions, enrollment, fee collection, grade processing and library records with a computerized Student Information Management System (SIMS). The current system causes data duplication, delayed results, inconsistent fee reconciliation and record misplacement. □filecite□turn0file0□L71-L84□

a. Major stakeholders and primary interactions

Stakeholder	Category	Primary interaction
Students	Primary external users	Apply for admission, enroll in courses, view fees/results, access library-related records.
Applicants	External users	Submit admission information and supporting details; receive admission status.
Admissions staff	Operational users	Verify applications, create student records, process admissions and enrollment.
Academic/Examination staff	Operational users	Enter/process grades, verify results and publish academic information.
Accounts/Finance staff	Operational users	Record fees, reconcile payments, issue receipts and monitor outstanding balances.
Librarians	Operational users	Manage library member/transaction records and circulation information.
Management	Management stakeholder	View reports, monitor operations and make decisions.
System administrator/IT staff	Technical stakeholder	Manage users, security, configuration, backups and system availability.

b. Suitable fact-finding techniques

- Interviews: conduct structured/semi-structured interviews with admissions, finance, examination, library staff, management and selected students to uncover detailed procedures, exceptions and problems.
- Questionnaires: distribute standardized questionnaires to a larger student/staff population to collect comparable information quickly and identify common pain points.
- Observation: observe admission, fee collection and record-handling activities directly to discover actual practices, delays, workarounds and duplicated data that users may forget to mention.
- Document sampling: examine admission forms, ledgers, fee receipts, grade sheets, enrollment forms and library records to identify data fields, formats, approvals and inconsistencies.
- Use the techniques together: interviews explain why a process exists, observation shows how it actually happens, questionnaires reveal patterns across many users, and document sampling verifies the data captured.

c. Context Diagram (Level 0) and Level 1 DFD

Level 0 Context Diagram

```
[Applicant/Student] -- admission/enrollment details -->
+-----+
| STUDENT INFORMATION |
| MANAGEMENT SYSTEM (SIMS) |
+-----+

[Applicant/Student] <-- admission status, timetable, fee/result info
[Admissions] -----> application verification / enrollment data
[Admissions] <----- admission & enrollment reports
[Finance] -----> payment/fee data
[Finance] <----- receipts, balances, reconciliation reports
[Academic/Exam] ----> grades/course data
[Academic/Exam] <---- result reports
[Library] -----> library transaction data
[Library] <----- student/member information and transaction status
[Management] <----- management reports / summaries
```

Level 1 — Admission and Fee Processing

```
Applicant -> (1.0 Process Admission) -> [D1 Student Records]
|
+--> Admission Status <---+

Student/Admissions -> (2.0 Process Enrollment) -> [D2 Enrollment]
|
+--> Enrollment Confirmation

Student -> (3.0 Process Fee Payment) -> [D3 Fee Records]
Finance/Bank -> payment verification -> (3.0)
(3.0) -> receipt / outstanding balance -> Student
(3.0) -> reconciliation report -> Finance
```

Key rule: Level 1 processes preserve the major external inputs/outputs shown in Level 0 (DFD balancing).

d. ER Diagram

Entity	Key / important attributes	Relationships
STUDENT	Student_ID (PK), Name, DOB, Address, Phone, Email	Student applies/enrolls; student has fee records; student has results; student may have library transactions.
APPLICATION	Application_ID (PK), Student_ID (FK), Application_Date, Program, Status	One student can have application records; application is processed by Admissions.

COURSE	Course_ID (PK), Course_Name, Credit_Hours	Course participates in enrollment and results.
ENROLLMENT	Enrollment_ID (PK), Student_ID (FK), Course_ID (FK), Semester, Date	Associative entity resolving Student–Course many-to-many relationship.
FEE_PAYMENT	Payment_ID (PK), Student_ID (FK), Amount, Date, Method, Status	Student has many payments.
RESULT	Result_ID (PK), Student_ID (FK), Course_ID (FK), Grade, Marks	Student receives results for courses.
LIBRARY_TRANSACTION	Transaction_ID (PK), Student_ID (FK), Book_ID (FK), Issue_Date, Return_Date	Student can have many library transactions.
BOOK	Book_ID (PK), Title, Author, Quantity	Book participates in library transactions.

Simplified ER relationship view

```

STUDENT 1 -----< APPLICATION
STUDENT 1 -----< ENROLLMENT >----- 1 COURSE
STUDENT 1 -----< FEE_PAYMENT
STUDENT 1 -----< RESULT >----- 1 COURSE
STUDENT 1 -----< LIBRARY_TRANSACTION > 1 BOOK
Notation: 1 = one; < = many.

```

18. Critically analyze the Systems Development Life Cycle (SDLC). Compare Waterfall, Prototyping, Spiral, and Incremental models in terms of risk management, cost, and requirement flexibility. Recommend and justify the most suitable model for developing an enterprise online banking portal. [15 Marks]

SDLC is a structured framework used to plan, analyze, design, develop, test, implement and maintain an information system. It provides control over requirements, quality, resources, documentation and change.

Model	Risk management	Cost pattern	Requirement flexibility
Waterfall	Risk is addressed mainly through planned phases and reviews; late changes can be costly.	Costs are planned relatively early; changes later can increase rework.	Low; requirements are expected to be defined early.
Prototyping	Reduces requirement/communication risk by letting users see an early model.	Prototype adds initial effort, but may reduce misunderstanding and rework.	High; feedback can reshape requirements.
Spiral	Explicitly emphasizes repeated risk identification, analysis and mitigation.	Potentially expensive because repeated risk analysis and iterations require resources.	High; requirements can evolve across cycles.
Incremental	Risk is distributed across increments; smaller releases can expose problems earlier.	Costs are distributed across releases; usable parts can be delivered earlier.	High; later increments can respond to changing priorities.

For an enterprise online banking portal, relevant selection factors include security, integration, reliability, regulatory/compliance requirements, complex requirements, testing, risk control and the need for controlled delivery. A risk-driven iterative approach can be used where high-risk architecture/security concerns are analyzed early, while increments deliver functionality progressively. In an examination answer, the choice should be justified against the stated organizational constraints rather than by claiming one model is universally best.

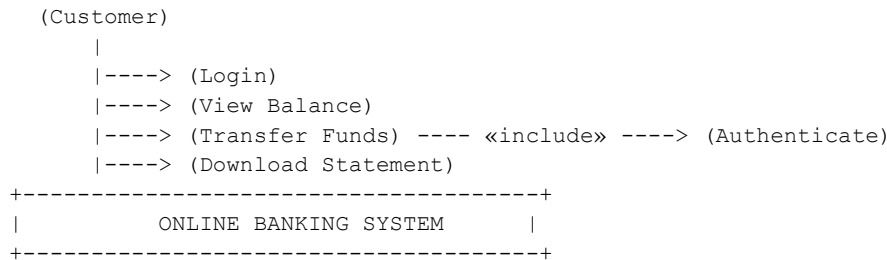
A practical approach can combine disciplined requirements and architecture with iterative risk-focused development: establish critical security and integration requirements early, prototype sensitive interfaces where necessary, develop functionality in controlled increments, perform extensive testing, and maintain traceable documentation and approvals.

19. Discuss Object-Oriented Analysis and Design (OOAD) using UML. Explain the structural purpose, components, and behavioral notation of Use Case, Class, Sequence, and State Transition Diagrams with appropriate diagrams. [15 Marks]

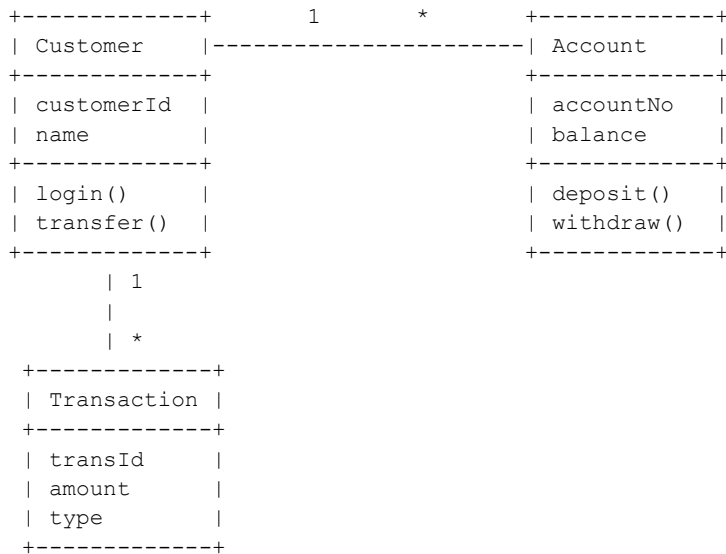
OOAD models a system as interacting objects/classes that contain data and behavior. UML provides standardized graphical notation for communicating these models.

Diagram	Main purpose	Important notation/components
Use Case	Captures functional requirements from an actor's perspective.	Actor, use case oval, system boundary, association, «include», «extend».
Class	Shows structural/static organization of objects.	Class name, attributes, operations, associations, multiplicity, inheritance/generalization.
Sequence	Shows time-ordered interaction between objects.	Lifeline, activation bar, messages, return messages, guards/conditions.
State Transition	Shows how an object changes state in response to events.	States, transitions, events, guards, actions, initial/final states.

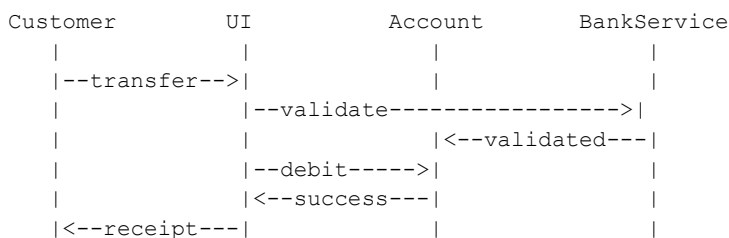
Use Case — Online Banking Example



Class Diagram — simplified



Sequence Diagram — Transfer



State Transition — Account

```
[Initial] -> Active
Active --withdraw request / valid--> Active
Active --balance reaches zero/limit--> Restricted
Restricted --funds added--> Active
Active --closure request--> Closed
Closed -> [Final]
```

20. Explain the essential activities executed during the System Implementation phase. Detail system installation strategies (Direct, Parallel, Pilot, Phased), Software Quality Assurance mechanisms (formal technical reviews, walkthroughs, inspections), and the four types of software maintenance. [15 Marks]

System Implementation converts the designed system into an operational system. Major activities include final development/configuration, testing, data conversion, installation, user training, documentation, change management, deployment, and post-installation support.

1. Finalize and configure software, hardware, database and network components.
2. Perform unit, integration, system and acceptance testing and correct defects.
3. Prepare and cleanse existing data; convert and migrate data into the new system.
4. Install software and infrastructure and configure security, access and operational settings.
5. Train users, administrators and support staff; provide manuals and operating procedures.
6. Choose and execute a conversion strategy: direct, parallel, pilot or phased.
7. Monitor the deployed system, resolve operational issues and conduct post-implementation review.

Installation strategies:

Strategy	Meaning	Main characteristic
Direct cutover	Old system is stopped and new system starts at a specified time.	Fast but higher operational risk because there is little fallback.
Parallel	Old and new systems operate together for a period.	Higher resource/workload, but old system provides a fallback and comparison.
Pilot	New system is introduced first in one selected site/group.	Limits initial exposure and allows learning before wider deployment.
Phased	System is introduced module-by-module or stage-by-stage.	Reduces change shock and spreads risk/work across stages.

SQA mechanisms:

- Formal Technical Review (FTR): structured review of a technical work product against requirements, standards, correctness and quality criteria.
- Walkthrough: author or team presents a work product to peers, who ask questions and identify defects, ambiguities or omissions.
- Inspection: a more formal, systematic defect-detection activity using defined roles, preparation, checklists and recorded findings.

Four types of software maintenance:

- Corrective: fixes errors or faults discovered after deployment.
- Adaptive: changes the system to work with changes in operating systems, hardware, regulations, interfaces or other environments.
- Perfective: improves performance, usability, maintainability, or adds/improves functionality based on user needs.

- Preventive: restructures or improves software internally to reduce the likelihood of future failures and make future maintenance easier.

MODEL SET 2 — COMPLETE ANSWERS

Set 2 covers TPS/MIS, stakeholders, DFDs, ER modeling, data dictionaries, structured logic, UML, feasibility, UI/input design, testing, plus hospital, OOAD, and SQA questions. □filecite□turn0file0□L115-L151□

1. Differentiate between a Transaction Processing System (TPS) and a Management Information System (MIS). [2 Marks]

TPS records and processes routine, day-to-day transactions such as sales, payments and registrations. MIS uses processed organizational data, often including TPS data, to produce summarized information and reports that support managerial monitoring and decision-making.

2. List any four key stakeholders involved in enterprise software development. [2 Marks]

Any four: end users/customers, business managers/sponsors, system analysts, software developers, database/network administrators, regulators, suppliers/partners, and project managers.

3. What is an external entity in a Data Flow Diagram? How is it symbolized? [2 Marks]

An external entity is a person, department, organization, or other system outside the system boundary that sends data to or receives data from the system. It is commonly represented by a rectangle.

4. Define cardinality and modality in Entity Relationship modeling. [2 Marks]

Cardinality specifies the maximum relationship occurrence, such as one-to-one, one-to-many, or many-to-many. Modality specifies whether participation is optional or mandatory, commonly represented as minimum participation such as 0 or 1.

5. Explain the concept of Data Store repository entries in a Data Dictionary. [2 Marks]

A data-store entry documents a repository of data used by the system, such as a file or database table. It records information such as store name, description, contents/data elements, and how the store is used by processes.

6. When is Structured English preferred over a Decision Table for logic modeling? [2 Marks]

Structured English is preferred when the logic is procedural or sequential, includes nested IF/ELSE statements or repetition, and is easier to describe as step-by-step processing than as combinations of many condition columns.

7. What is a Use Case Actor? Give one example. [2 Marks]

A Use Case Actor is an external role that interacts with a system to achieve a goal. Example: Customer is an actor in an online banking system.

8. Define Payback Period in economic feasibility assessment. [2 Marks]

Payback Period is the amount of time required for cumulative cash benefits/savings from a project to recover its initial investment. A shorter payback period generally means the initial investment is recovered sooner.

9. State two primary objectives of effective user interface and input form design. [2 Marks]

Two objectives are to make interaction easy and understandable for users and to minimize input errors while ensuring accurate, complete and valid data entry.

10. Distinguish between Alpha Testing and Beta Testing. [2 Marks]

Alpha testing is internal testing performed by the development organization in a controlled environment. Beta testing is performed by selected real users/customers in a more realistic environment before wider release.

SECTION B — SHORT ANSWERS

11. Examine the evolving role of a System Analyst in modern organizations. Discuss how their communication and analytical skills bridge business requirements and technical implementation. [7 Marks]

The modern System Analyst is not limited to documenting an existing process. The role increasingly involves business analysis, digital transformation, integration, data understanding, risk awareness, user experience, and coordination among technical and non-technical stakeholders.

1. Business understanding: understands organizational goals, workflows, constraints and performance needs.
2. Requirement analysis: gathers, validates, prioritizes and documents functional and non-functional requirements.
3. Technical translation: converts business needs into models, specifications, interfaces, data structures and constraints that developers can implement.
4. Communication: explains technical concepts in understandable language and translates user concerns into actionable technical requirements.
5. Conflict resolution: identifies conflicting stakeholder expectations and helps negotiate consistent requirements.
6. Analytical problem solving: compares alternatives, identifies root causes, evaluates feasibility and assesses impacts of proposed changes.
7. Traceability and validation: connects requirements to models, tests and delivered functionality so that implementation can be checked against business needs.

12. Explain the rules and conventions governing DFD construction. What constitutes “Black Hole,” “Miracle,” and “Gray Hole” errors in process modeling? [7 Marks]

DFD construction conventions include:

1. Every process should have at least one input and at least one output.
2. Data flows should be named meaningfully.
3. A process should transform incoming data into outgoing data.
4. Data should not flow directly from one external entity to another without a process where system processing is required.
5. Data should not normally flow directly between two data stores without a process.
6. External entities represent sources/destinations outside the system boundary.
7. DFD levels should be balanced so parent inputs/outputs remain represented in child diagrams.
8. Processes should be numbered and decomposition should remain logically consistent.

Errors:

- Black Hole: a process has input data but produces no output. It “absorbs” data without showing a result.
- Miracle: a process produces output without any input. It appears to create information from nothing.
- Gray Hole: a process has inputs and outputs, but the outputs cannot logically be derived from the inputs shown, indicating incomplete or incorrect process logic.

13. Discuss the concept of Data Dictionaries. Explain how data structures, data elements, and data stores are formally documented. [7 Marks]

A Data Dictionary is a centralized repository of metadata and definitions about the data used in an information system. It provides a common vocabulary and supports consistency among analysis and design models.

Data element documentation normally includes:

- Name/identifier
- Meaning/description
- Data type
- Length/size
- Allowed values/range
- Validation rules and aliases where applicable.

Data structure documentation describes a group of related data elements. For example:

STUDENT = Student_ID + Name + DOB + Address + Phone + Email.

It defines the components and their relationships/order.

Data store documentation describes a persistent collection such as a database table/file. It can specify store name, purpose, included data elements, key fields, source/processes that update it, and processes that read it.

The Data Dictionary improves consistency because analysts and developers use the same definitions, reduces ambiguity, supports impact analysis and helps maintain traceability between DFDs, ERDs, database design, forms, reports and programs.

14. Construct a Decision Table for a retail discount policy: “Give 15% discount if purchase > \$500 and member; 10% if purchase > \$500 but non-member; 5% if member with purchase ≤ \$500; else no discount.” [7 Marks]

The conditions are: purchase greater than \$500? and member? The actions are the four possible discount outcomes.

Condition / Action	R1	R2	R3	R4
Purchase > \$500?	Yes	Yes	No	No
Member?	Yes	No	Yes	No
Discount	15%	10%	5%	0%

15. Explain requirement gathering techniques including Joint Application Development (JAD) and rapid questionnaires. Discuss their comparative merits. [7 Marks]

Requirement gathering identifies what stakeholders need the proposed system to do and the constraints under which it must operate.

JAD is a facilitated, structured workshop in which users, managers, analysts and relevant technical participants work together to define requirements and resolve issues.

Advantages of JAD:

- Direct stakeholder participation.
- Faster clarification of conflicting requirements.
- Immediate discussion and consensus building.
- Useful for complex business processes.

Limitations: requires participant availability, skilled facilitation and careful preparation.

Rapid questionnaires are structured sets of questions distributed to many users. They can gather comparable information quickly and at relatively low cost.

Advantages:

- Reach many participants.
- Standardized answers are easier to compare.
- Useful for collecting broad opinions and identifying common problems.

Limitations:

- Limited depth.
- Ambiguous questions may produce poor answers.
- Respondents may omit important exceptions.

Comparison: JAD is richer and more interactive but resource-intensive; questionnaires are faster and broader but less detailed. Combining methods often gives stronger requirements coverage.

16. Discuss the differences between Logical Design and Physical Design in database modeling and input/output structure architecture. [7 Marks]

Logical Design describes the system independent of particular implementation technology. It defines data entities, attributes, relationships, processing requirements, logical inputs/outputs and business rules.

Physical Design specifies how the logical design will actually be implemented.

Database distinction:

- Logical: entities, attributes, keys, relationships and normalization.
- Physical: tables, indexes, storage structures, data types, partitions and DBMS-specific configuration.

Input/output distinction:

- Logical input/output identifies what information is required and what information must be produced.
- Physical input/output specifies actual forms, screens, devices, file layouts, report formats and technical interfaces.

Logical design answers “what and how logically”; physical design answers “how in the chosen technology and operating environment.” Physical design must implement the logical requirements while considering performance, security, hardware, software and operational constraints.

SECTION C — LONG ANSWERS

17. COMPULSORY CASE STUDY — HOSPITAL INFORMATION & APPOINTMENT SYSTEM [15 Marks]

The hospital is moving from paper scheduling and manual records to a centralized HIAS. Patients register and book appointments; doctors manage schedules, EMRs and prescriptions; the laboratory uploads reports; accounts manages billing and insurance claims. □filecite□turn0file0□L167-L179□

a. External entities and requirements

External entity	Main interaction
Patient	Registration, doctor search, appointment booking, personal record access, payment information.
Doctor	View schedule, update EMR, issue electronic prescriptions.
Laboratory	Upload investigation/lab reports to patient records.
Accounts/Finance	Billing, payment processing, insurance claim management.
Insurance provider	Claim/coverage information and claim responses.
Hospital administration	Reports, monitoring and management information.

Functional requirements:

- Patient registration and authentication.
- Doctor schedule search and appointment booking/cancellation.
- Doctor schedule management and daily appointment viewing.
- Electronic medical record creation/update and retrieval.
- Electronic prescription creation and access.
- Laboratory report upload and association with patient records.
- Automated billing and insurance claim processing.
- Management/reporting functions.

Non-functional requirements:

- Security and privacy: protect patient information through authentication, authorization, secure communication and access controls.
- Availability/performance: provide responsive appointment and record services with reliable availability.
- Data integrity: ensure records, prescriptions, reports and billing information remain accurate and consistent.

b. Level 0 Context Diagram

HIAS Context Diagram

```

Patient ----- registration / booking / data ----->
                +-----+
                |          HIAS          |
                +-----+
Patient <----- confirmations / schedules / records
Doctor ----- schedule/EMR/prescription data -----> HIAS
Doctor <----- appointments / patient records
Laboratory ---- investigation reports -----> HIAS
Accounts ----- billing/claim data -----> HIAS
Insurance <---- claim requests / responses -----> HIAS
Administration <---- management reports -----> HIAS

```

c. Level 1 DFD — Appointment Scheduling and Electronic Prescription

Level 1 DFD

```

Patient -> (1.0 Register/Search Doctor) -> [D1 Patient Records]
Patient -> (2.0 Check Availability) <-> [D2 Doctor Schedule]
Patient -> (3.0 Book Appointment) -> [D3 Appointment Records]
(3.0) -> Appointment Confirmation -> Patient

Doctor -> (4.0 View Appointment) <- [D3 Appointment Records]
(4.0) -> Daily Schedule -> Doctor
Doctor -> (5.0 Create Prescription) -> [D4 Prescription Records]
(5.0) <- Patient EMR from [D1/D5 EMR]
(5.0) -> Electronic Prescription -> Patient/Pharmacy interface

```

d. Structured feasibility assessment

Area	Assessment points	Conclusion criteria
Technical	Existing infrastructure, secure database, network, integration with laboratory/accounting/insurance systems, staff skills, backup/security capability.	Viable if required infrastructure, integration and security capability can be provided.
Operational	Workflow fit, staff training, patient usability, privacy procedures, change management and user acceptance.	Viable if stakeholders accept the new workflow and training/support are adequate.
Economic	Development/licensing/hardware/training/maintenance costs versus reduced paperwork, improved scheduling, lower duplication and administrative efficiency.	Viable if measurable and strategic benefits justify total cost.
Schedule	Development, migration, testing, training and deployment time against hospital deadline and resource availability.	Viable if milestones can be achieved without unsafe compression of testing/training.

18. Explain the concept of software architecture modularity. Critically discuss the spectrum of Cohesion (from Coincidental to Functional) and Coupling (from Data to Content). Explain how high cohesion and loose coupling contribute to maintainability and fault isolation. [15 Marks]

Modularity divides a system into relatively independent modules with defined responsibilities and interfaces. Good modular architecture makes individual parts easier to understand, develop, test, replace and maintain.

Cohesion spectrum (weak to strong):

Cohesion	Meaning
Coincidental	Unrelated tasks are grouped together with little logical connection.
Logical	Related by category/type, but one of several different functions may be selected.
Temporal	Tasks are grouped because they occur at the same time, such as initialization.
Procedural	Elements are grouped because they follow a particular execution sequence.
Communicational	Operations use the same data or contribute to the same output.
Sequential	Output from one part becomes input to another within the same module.
Functional	All elements contribute to one well-defined function; generally the strongest desirable cohesion.

Coupling spectrum (with data coupling as the desirable loose end in the question):

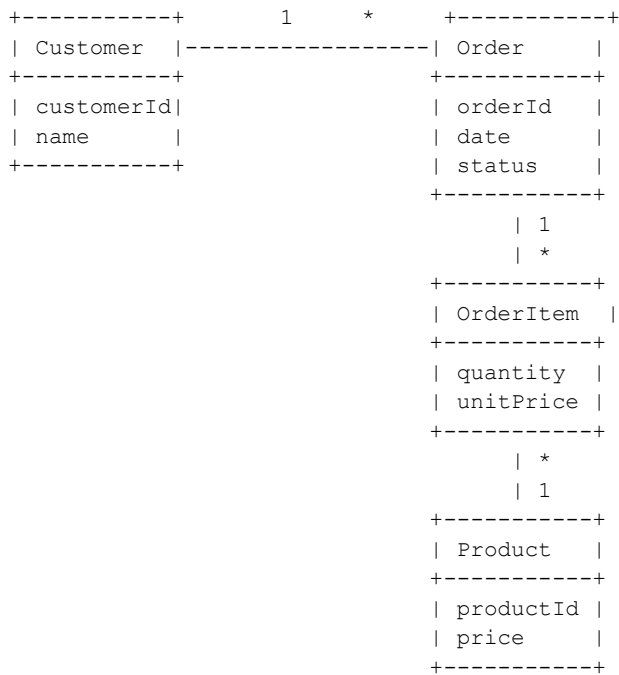
Coupling	Meaning
Data	Modules communicate by passing only required data parameters.
Stamp	A composite structure is passed, even though only part may be needed.
Control	One module passes control information that determines another module's logic.
Common	Modules share common/global data.
Content	One module directly accesses or modifies another module's internal contents; very strong dependency.

High cohesion means a module has a focused responsibility. If one module changes, unrelated modules are less likely to be affected. Loose coupling means modules depend on each other through small, stable interfaces. Together they improve maintainability, testing, reuse, change isolation and fault isolation. A defect in one highly cohesive module with loose dependencies is less likely to propagate across the system.

19. Discuss object modeling with Unified Modeling Language (UML). Explain the distinction between structural and behavioral models. Draw and explain a comprehensive Class Diagram and an accompanying Sequence Diagram for an Online Shopping Cart checkout process. [15 Marks]

UML is a standardized modeling language for visualizing, specifying and documenting software systems. Structural models describe what objects/classes exist and how they are related; behavioral models describe how those objects interact or change over time.

Class Diagram — Online Shopping Checkout

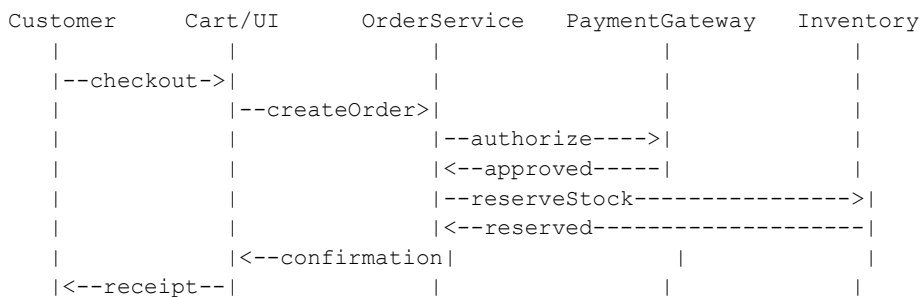


```

Order --> PaymentService --> Payment
Order --> InventoryService --> Inventory

```

Sequence Diagram — Checkout



The class diagram captures the static entities and relationships, while the sequence diagram shows the time-ordered checkout interaction and responsibility handoff.

20. Define System Testing. Differentiate systematically between Unit Testing, Integration Testing, System Testing, and Acceptance Testing (UAT). Explain the role of regression testing and formal technical reviews (FTR) in maintaining software quality assurance. [15 Marks]

System Testing is testing of the complete integrated software system to determine whether it satisfies specified functional and non-functional requirements in an environment representative of actual operation.

Testing level	Focus	Typical responsibility/objective
Unit Testing	Individual module/class/function.	Find local coding/logic defects early.
Integration Testing	Interfaces and interactions among modules/subsystems.	Find interface, data-transfer and interaction defects.
System Testing	Complete integrated system.	Verify end-to-end behavior against requirements.
Acceptance Testing (UAT)	Business/user acceptance of the system.	Confirm that the system is acceptable for operational use and meets user/business needs.

Regression testing reruns relevant existing tests after changes to verify that previously working functionality has not been broken by modifications, bug fixes, integrations or enhancements.

Formal Technical Reviews support SQA by examining work products such as requirements, designs, code or test artifacts against standards and quality criteria. They detect defects early, improve consistency, provide documented findings, and reduce the cost of discovering errors later. FTRs complement testing because reviews can identify defects in artifacts before executable software exists.

MODEL SET 3 — COMPLETE ANSWERS

Set 3 focuses on SDLC limitations, system boundary, requirements, ERD, CASE, decision logic, UML state machines, ROI, structure charts, maintenance, and three detailed case/design questions.

□filecite□turn0file0□L209-L244□

1. State two main limitations of the traditional Waterfall Model of SDLC. [2 Marks]

Two limitations are: (1) requirements are expected to be stable and changes later can cause substantial rework; and (2) working software and user feedback generally arrive late, so misunderstandings may be discovered after significant effort.

2. What is meant by an Information System's boundary and environment? [2 Marks]

The system boundary defines what is inside the system being analyzed and what is outside it. The environment consists of external people, organizations, systems, rules, and conditions that interact with or influence the system.

3. Differentiate between functional requirements and non-functional requirements. [2 Marks]

Functional requirements specify what the system must do, such as registering a customer or generating an invoice. Non-functional requirements specify quality or constraints, such as performance, security, usability, availability, and reliability.

4. What is a recursive relationship in an Entity Relationship Diagram? [2 Marks]

A recursive relationship is a relationship in which an entity is related to itself. Example: an Employee supervises another Employee, where both relationship roles refer to the Employee entity.

5. Define Upper-CASE vs. Lower-CASE tools in system modeling. [2 Marks]

Upper-CASE tools mainly support early SDLC activities such as planning, requirements, analysis and design. Lower-CASE tools mainly support later activities such as implementation, testing, configuration and maintenance.

6. Mention the purpose of an Extended Entry Decision Table. [2 Marks]

An Extended Entry Decision Table represents condition/action rules where condition entries can use more detailed values or ranges rather than only simple yes/no entries, allowing richer business-rule representation.

7. State the purpose of a State Machine (State Transition) Diagram in UML. [2 Marks]

It models the dynamic behavior of an object/system by showing its possible states and transitions caused by events or conditions.

8. What is Return on Investment (ROI) and how is it expressed mathematically? [2 Marks]

ROI measures the return generated relative to an investment. A common expression is $ROI = (\text{Net Benefit or Return} \div \text{Investment Cost}) \times 100\%$.

9. What is a Structure Chart and what do arrows with open/solid circles indicate? [2 Marks]

A Structure Chart is a hierarchical model showing system modules and their calling relationships. In common structure-chart notation, arrows show module/data/control relationships, while open/solid circles can be used to indicate different types of data/control passing depending on the notation standard used by the course.

10. Define preventive software maintenance. [2 Marks]

Preventive maintenance modifies or restructures software to reduce the probability of future failures and improve maintainability, even when no immediate fault requires correction.

SECTION B — SHORT ANSWERS

11. Explain the major phases of the Systems Development Life Cycle (SDLC) and discuss the primary deliverables produced at the end of each phase. [7 Marks]

The SDLC organizes system development into phases. Names can vary by methodology, but the major activities are:

1. Planning/Preliminary Investigation: identify the problem, objectives, scope, stakeholders and initial constraints. Deliverable: preliminary study/project plan.
2. Requirements/Analysis: investigate the existing system and gather, validate and document requirements. Deliverables: requirements specification, process/data models, feasibility findings.
3. System Design: convert requirements into logical and physical design specifications. Deliverables: architecture, database design, interface/input/output design, detailed specifications.
4. Development/Construction: build/configure software, database and interfaces. Deliverable: implemented software components and technical documentation.
5. Testing: verify units, integrations and complete system against requirements. Deliverables: test plans, test cases/results, defect records and acceptance evidence.
6. Implementation/Conversion: install, migrate data, train users and deploy using a selected conversion strategy. Deliverables: operational system, migrated data, training/operating materials.
7. Maintenance: correct, adapt, perfect and prevent future problems. Deliverables: maintenance releases, change records, updated documentation.

12. Explain the procedure for drawing an Entity Relationship Diagram (ERD). Describe entity types, weak entities, attributes, and key constraints with examples. [7 Marks]

Procedure:

1. Identify major entities from requirements.
2. Identify attributes describing each entity.
3. Select candidate/primary keys.
4. Identify relationships among entities.
5. Determine cardinalities and participation/modality.
6. Resolve many-to-many relationships using associative entities where required.
7. Identify weak entities and their identifying relationships.
8. Review the ERD for completeness, consistency and alignment with requirements.

Entity types:

- Strong entity: has its own key, e.g., STUDENT(Student_ID, Name).
- Weak entity: cannot be uniquely identified by its own attributes alone and depends on an owner entity, e.g., DEPENDENT may be identified using Employee_ID plus Dependent_Name.

Attributes may be simple, composite, multivalued or derived. Example: Address may be composite (Street, City, PostalCode).

Key constraints identify unique records. A primary key uniquely identifies each entity occurrence. Foreign keys connect related entities in relational implementation.

Example:

CUSTOMER 1 ----< ORDER

Customer_ID is PK in CUSTOMER; Order_ID is PK in ORDER; Customer_ID may appear as FK in ORDER.

13. Describe the structure and management of a Project Repository. Explain how Data Dictionaries enforce consistency across analysis and design models. [7 Marks]

A Project Repository is a centralized location for system-development artifacts and metadata. It may contain requirements, process models, DFDs, ERDs, UML models, data definitions, design specifications, test artifacts, standards, decisions and version information.

Management involves:

1. Controlled naming and definitions.
2. Version and change control.
3. Access permissions.
4. Backup and recovery.
5. Linking/traceability between artifacts.
6. Consistency and impact checking.
7. Controlled updates by authorized team members.

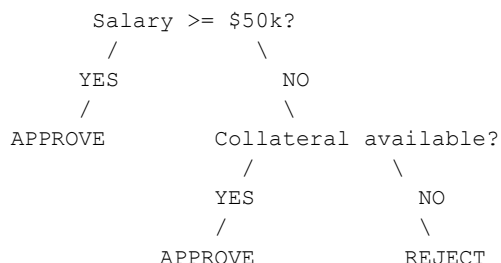
A Data Dictionary enforces consistency by defining common names, meanings, types, lengths, structures and constraints for data elements, stores and flows. For example, if Student_ID is defined as a particular identifier in the Data Dictionary, the same definition can be used in DFD data flows, ERD attributes, forms, database design and program specifications. This reduces ambiguity, duplication and mismatched data definitions across models.

14. Differentiate between Decision Tables and Decision Trees. Draw a Decision Tree to represent loan eligibility: Salary $\geq$ \$50k grants approval; Salary $<$ \$50k with collateral grants approval; otherwise reject. [7 Marks]

A Decision Table represents combinations of conditions and corresponding actions in rows/columns, making it useful for exhaustive rule coverage. A Decision Tree represents decision logic as branches, making sequential alternatives easy to visualize.

Decision Tree:

Loan Eligibility Decision Tree



15. Explain the concepts of Tangible vs. Intangible Costs and Benefits. Provide two distinct real-world examples for each category in corporate systems. [7 Marks]

Tangible costs/benefits can be measured relatively directly in monetary or physical terms. Intangible costs/benefits are real but difficult to quantify precisely.

Tangible costs:

1. Purchase of servers and networking equipment for an enterprise system.
2. Employee training fees or software licensing expenses.

Tangible benefits:

1. Reduction in paper and printing expenses after digitizing records.
2. Reduction in staff processing hours due to workflow automation.

Intangible costs:

1. Temporary employee stress or resistance during a major system change.
2. Disruption to organizational routines during transition.

Intangible benefits:

1. Improved customer satisfaction due to faster service.
2. Better management decision quality due to more timely information.

Both categories should be considered because excluding intangible effects can give an incomplete picture of project impact.

16. Discuss Input and Form Design techniques. What design principles should be followed to ensure data integrity, user ergonomics, and minimal data entry error? [7 Marks]

Input/form design determines how users enter data and directly affects accuracy, speed and user experience.

Key principles:

1. Simplicity: request only necessary information.
2. Logical layout: group related fields and follow a natural sequence.
3. Clear labels: use understandable, consistent field names and instructions.
4. Appropriate controls: use drop-down lists, radio buttons, checkboxes, date pickers and constrained fields where appropriate.
5. Validation: apply type, range, format, completeness and consistency checks.
6. Defaults: provide safe, useful defaults when appropriate.
7. Error prevention: prevent invalid entries before submission.
8. Error messages: identify the problem and explain how to correct it.
9. Consistency: keep layouts, terminology and controls consistent.
10. Accessibility/ergonomics: readable text, sensible navigation, keyboard support and manageable cognitive load.
11. Confirmation: clearly show important submitted values and provide confirmation for critical transactions.
12. Data integrity: use database constraints and transaction controls in addition to interface validation.

SECTION C — LONG ANSWERS

17. COMPULSORY CASE STUDY — E-COMMERCE RETAIL & INVENTORY MANAGEMENT [15 Marks]

The scenario describes an integrated E-Commerce & Warehouse Inventory Platform supporting customer accounts, catalog browsing, secure third-party payment, real-time inventory decrementing, supplier

purchase orders at reorder points, dispatch manifests, shipping tracking and returns/RMA processing.

□filecite□turn0file0□L246-L270□

a. External entities and Context Diagram

External entity	Data sent to system	Data received
Customer	Account data, searches, order details, payment request, return request	Catalog, order confirmation, payment status, shipment tracking, return status
Payment Gateway	Payment verification/authorization response	Payment request/details
Supplier	Supplier/product/PO acknowledgement data	Purchase orders
Logistics/Dispatch	Dispatch information, shipping updates, returns data	Manifests, shipping/return instructions
Shipping/Tracking Service	Tracking events/status	Shipment/tracking request
Management	Reporting requests	Sales, inventory and operational reports

Level 0 Context Diagram

```

Customer ----- orders/account/returns ----->
      +-----+
      | E-COMMERCE & WAREHOUSE |
      | INVENTORY PLATFORM    |
      +-----+
Customer <----- catalog/order/payment/tracking
Payment Gateway <---- payment verification -----> System
Supplier <----- purchase orders -----> System
Logistics <----- manifests/returns -----> System
Shipping Service <---- tracking requests/status -----> System
Management <----- reports -----> System

```

b. Level 1 DFD

Level 1 — Order Processing, Payment Verification, Inventory Replenishment

```

Customer -> (1.0 Order Processing) <-> [D1 Orders]
(1.0) -> order items -> (2.0 Payment Verification)
(2.0) <-> Payment Gateway
(2.0) -> verified payment -> (3.0 Inventory Update)
(3.0) <-> [D2 Product_Inventory]
(3.0) -> confirmed order -> [D1 Orders]

[D2 Product_Inventory] -> stock level -> (4.0 Replenishment)
(4.0) -> Purchase Order -> Supplier
Supplier -> acknowledgement -> (4.0)
(4.0) -> [D3 Purchase Orders]
(1.0) -> dispatch request -> Logistics
Logistics -> tracking/dispatch updates -> (1.0)

```

c. ER Diagram

Entity	Key attributes	Relationships/cardinality
CUSTOMER	Customer_ID (PK), Name, Email, Phone, Address	1 Customer places many Orders (1:M).
ORDER	Order_ID (PK), Customer_ID (FK), Order_Date, Status, Total	Each Order belongs to one Customer; an Order has many Order_Items.
ORDER_ITEM	OrderItem_ID (PK), Order_ID (FK), Product_ID (FK), Quantity, Unit_Price	Many items belong to one Order; each item references one Product.
PRODUCT	Product_ID (PK), Name, Price, SKU, Reorder_Point	One Product can occur in many Order_Items; Product has inventory information.

PRODUCT_INVENTORY	Product_ID (PK/FK), Stock_Qty, Reorder_Point, Last_Updated	Associated with Product; stock changes through sales/replenishment.
SUPPLIER	Supplier_ID (PK), Name, Contact, Address	Supplier receives many Purchase Orders.
PURCHASE_ORDER	PO_ID (PK), Supplier_ID (FK), PO_Date, Status	Many POs can be issued to one Supplier.
SHIPMENT	Shipment_ID (PK), Order_ID (FK), Carrier, Tracking_No, Status	An Order may have shipment(s), depending on business rule.

ER relationship view

```

CUSTOMER 1 ----< ORDER 1 ----< ORDER_ITEM >---- 1 PRODUCT
PRODUCT 1 ---- 1 PRODUCT_INVENTORY
SUPPLIER 1 ----< PURCHASE_ORDER
ORDER 1 ----< SHIPMENT

```

d. Data Dictionary entry for D1: Product_Inventory

Field	Type/example	Description / validation
Product_ID	Integer/VARCHAR	Unique product identifier; PK/FK to Product.
Stock_Qty	Integer	Current available inventory quantity; non-negative.
Reorder_Point	Integer	Threshold at which replenishment is triggered.
Last_Updated	DateTime	Time of most recent inventory update.
Reserved_Qty	Integer (if used)	Quantity reserved for confirmed orders; non-negative.
Available_Qty	Derived Integer	Stock available for sale after applicable reservations.

The Data Dictionary should also document the store purpose, source processes, processes that read/update it, security/access rules, relationships to other stores, and business rules such as triggering replenishment when stock reaches the defined reorder point.

18. Discuss Project Scheduling and Feasibility Analysis in System Planning. Explain the use of Gantt Charts and PERT/CPM networks in monitoring project milestones. Given a hypothetical system development project, explain how the Critical Path is determined and managed. [15 Marks]

Project scheduling converts project scope into activities, dependencies, durations, resources and milestones. Feasibility analysis determines whether the project is practical from technical, operational, economic and schedule perspectives.

Gantt Chart: a time-scaled bar chart showing activities against calendar periods. It helps show start/end dates, overlaps, progress, milestones and delays.

PERT/CPM: network-based scheduling techniques that represent activities and dependencies. CPM focuses on activity durations and critical path; PERT can incorporate uncertainty using estimated activity times.

Activity	Duration	Predecessor
A Requirements	3 days	—
B Analysis/Design	4 days	A
C Development	6 days	B
D Testing	3 days	C
E Training	2 days	D
F Deployment	1 day	E

Hypothetical dependency network

```
START -> A(3) -> B(4) -> C(6) -> D(3) -> E(2) -> F(1) -> FINISH
```

Total path = 3 + 4 + 6 + 3 + 2 + 1 = 19 days.

Because this simple network contains one complete path, that path is the Critical Path.

Critical Path determination:

8. List all activities and their durations.
9. Identify precedence/dependency relationships.
10. Draw the network.
11. Calculate earliest start/finish times from the beginning (forward pass).
12. Calculate latest start/finish times from the end (backward pass).
13. Compute float/slack. Activities with zero total float lie on the critical path.
14. Monitor critical activities closely because delay in one can delay the project completion date.

Managing the critical path involves tracking actual progress, identifying schedule variance, resolving resource constraints, revising estimates when justified, and considering controlled schedule recovery actions where appropriate.

19. Explain the principles of Database Design in system engineering. Discuss the step-by-step transition from an unnormalized data set through First Normal Form (1NF), Second Normal Form (2NF), and Third Normal Form (3NF). Why is normalization indispensable for enterprise systems? [15 Marks]

Database design organizes data so that it is accurate, consistent, efficient and usable. Major principles include identifying entities and attributes, selecting keys, defining relationships, enforcing integrity constraints, reducing unnecessary duplication, considering security/performance, and aligning the database with business rules.

Normalization is a structured method of reducing redundancy and update anomalies.

Stage	Example and transformation
UNF	ORDER(OrderNo, CustomerName, {ProductID, ProductName, Qty, Price}) where repeating product groups occur.
1NF	Remove repeating groups so each row contains atomic values: ORDER_ITEM(OrderNo, ProductID, ProductName, Qty, Price). Composite key may be (OrderNo, ProductID).
2NF	Remove partial dependencies on part of a composite key. ProductName and Price depend on ProductID, not the whole (OrderNo, ProductID), so separate PRODUCT(ProductID, ProductName, Price) and keep ORDER_ITEM(OrderNo, ProductID, Qty).
3NF	Remove transitive dependencies. For example, if CustomerID determines CustomerName and CustomerAddress, keep those in CUSTOMER(CustomerID, CustomerName, CustomerAddress) rather than repeating them in ORDER.

Why normalization is important:

- Reduces unnecessary data redundancy.
- Prevents update anomalies, such as changing the same fact in many rows.
- Reduces insertion and deletion anomalies.
- Improves consistency and integrity.
- Makes relationships and ownership of data clearer.
- Supports maintainability as enterprise data grows.
- Provides a sound logical structure before physical optimization.

Normalization does not mean every database must be maximally normalized for every workload; physical design may later use carefully justified optimization. The essential exam point is that 1NF removes

repeating/non-atomic groups, 2NF removes partial dependencies, and 3NF removes transitive dependencies.

20. Explain the System Conversion and Installation phase. Compare Direct Cutover, Parallel Operation, Pilot Operation, and Phased Rollout in terms of operational risk, resource overhead, and failure recovery. Discuss post-implementation review procedures. [15 Marks]

System conversion and installation move the organization from the existing system to the new system. Activities include preparing infrastructure, installing/configuring software, migrating and validating data, training users, selecting a conversion strategy, executing deployment, monitoring operations and providing support.

Strategy	Operational risk	Resource overhead	Failure recovery
Direct Cutover	Higher because old system stops when new system starts.	Lower duration/overlap cost.	Limited immediate fallback unless backup/rollback is prepared.
Parallel	Lower because old system remains available.	High because both systems must be operated and reconciled.	Strong fallback to old system during overlap.
Pilot	Risk limited to selected site/group initially.	Moderate; requires pilot setup and evaluation.	Problems can be contained before full rollout.
Phased	Spread across modules/stages, limiting exposure per stage.	Moderate; multiple transition stages require planning.	Failures can often be isolated to the affected phase and rollback/recovery can be targeted.

Post-implementation review procedures:

15. Confirm whether objectives and requirements were achieved.
16. Review system performance, reliability, security and usability.
17. Compare actual costs, benefits and schedule against the original plan.
18. Collect feedback from users, management and support personnel.
19. Review defects, incidents, help-desk issues and unresolved requirements.
20. Check data migration accuracy and operational controls.
21. Document lessons learned and recommendations for future changes.
22. Create or update the maintenance/change backlog and ensure support ownership is clear.

MODEL SET 4 — COMPLETE ANSWERS

Set 4 covers ESS, DFD flows/balancing, conceptual data modeling, metadata, decision tables, UML generalization, operational feasibility, coupling, structure charts, maintenance, SQA, Agile/Spiral, UML interaction, and UI/output design. □filecite□turn0file0□L300-L334□

1. What is an Executive Support System (ESS) and who are its primary organizational users? [2 Marks]

An Executive Support System (ESS) provides senior executives with summarized, high-level information, trends, dashboards and reports to support strategic monitoring and decision-making. Its primary users are top-level executives and senior management.

2. Explain the concept of “Data Flow Splitting” and “Data Flow Merging” in DFDs. [2 Marks]

Data flow splitting occurs when one incoming data flow is distributed to multiple processes/destinations. Data flow merging occurs when multiple flows are combined into a single flow/process input when logically appropriate.

3. What is a composite attribute in conceptual data modeling? Give an example. [2 Marks]

A composite attribute can be divided into smaller meaningful sub-attributes. Example: Address can consist of Street, City, Province and PostalCode.

4. What is metadata and how is it documented within a data dictionary? [2 Marks]

Metadata is data about data, such as a field's name, meaning, type, size, allowed values and constraints. A Data Dictionary documents these definitions so models and implementations use consistent data specifications.

5. State the meaning of “Indifference Condition” in a Decision Table. [2 Marks]

An indifference condition means the value of a condition does not affect the action for a particular rule, so it can be represented by a “don't care” symbol/entry such as “–”.

6. Define generalization and specialization in UML Class Diagrams. [2 Marks]

Generalization represents a parent-child relationship where a more general superclass is shared by specialized subclasses. Specialization defines more specific subclasses from a general class, inheriting common attributes/operations.

7. What is Operational Feasibility and why is user acceptance central to it? [2 Marks]

Operational feasibility assesses whether the proposed system will work effectively within the organization's people, procedures and environment. User acceptance is central because a technically working system may fail operationally if users cannot or will not adopt it.

8. Define stamp coupling and control coupling. [2 Marks]

Stamp coupling occurs when modules pass a composite data structure and a receiving module uses only part of it. Control coupling occurs when one module passes control information that determines another module's behavior.

9. What is a Structure Chart “Module Call”? [2 Marks]

A module call is the invocation of one module by another module in a structure chart, representing the hierarchical control relationship between caller and called module.

10. Distinguish between Adaptive Maintenance and Perfective Maintenance. [2 Marks]

Adaptive maintenance changes software to operate in a changed environment, such as a new OS, hardware platform or external regulation. Perfective maintenance improves performance, usability, maintainability or functionality in response to user/business needs.

SECTION B — SHORT ANSWERS

11. Compare Structured Analysis & Design (SAD) with Object-Oriented Analysis & Design (OOAD). Discuss their foundational modeling paradigms. [7 Marks]

Structured Analysis & Design (SAD) and OOAD both analyze requirements and design systems but organize the problem differently.

SAD:

- Process/data centered.
- Uses DFDs to model data movement and transformation.
- Uses ERDs for data modeling.
- Uses decision tables/trees and Structured English for logic.
- Design commonly emphasizes functional decomposition and modules.

OOAD:

- Object/class centered.
- Combines data and behavior into objects/classes.
- Uses UML diagrams such as Use Case, Class, Sequence and State diagrams.
- Models relationships, interactions and object responsibilities.
- Supports inheritance, encapsulation and polymorphism in object-oriented designs.

Foundational paradigm: SAD decomposes processes and data flows; OOAD models collaborating objects that represent entities/responsibilities in the problem domain. Neither is inherently appropriate for every context; selection depends on system characteristics, team practices and required models.

12. Explain the concept and significance of DFD Balancing (level-to-level child-parent data flow consistency) with an illustrative diagram. [7 Marks]

DFD balancing means that the inputs and outputs of a parent process at one level are preserved when that process is decomposed into a lower-level child diagram. Internal detail may be added, but the external data interface must remain logically consistent.

Significance:

1. Prevents information from appearing/disappearing between levels.
2. Maintains traceability from high-level to detailed models.
3. Helps detect missing processes or incorrect flows.
4. Improves communication and model reliability.

Example:

Balanced DFD example

Parent Level:	Child Level:
+-----+	+-----+
2.0 Process	2.1 Validate

```

|           |           | 2.2 Store           |
+-----+           | 2.3 Confirm           |
IN: OrderData -----> 2.1
OUT: Confirmation <----- 2.3
The same external input/output must be represented after decomposition.

```

13. Discuss the role of CASE tools in repository management, diagram generation, code skeleton generation, and traceability throughout the SDLC. [7 Marks]

CASE tools support system development through automation and centralized project information.

1. Repository management: store models, definitions, requirements, design objects and metadata in a shared repository.
2. Diagram generation: create/update DFDs, ERDs, UML diagrams and other models consistently.
3. Code skeleton generation: some tools transform design models into initial program/database structures, reducing repetitive work.
4. Traceability: link requirements to models, design elements, code components and tests.
5. Consistency: common repositories and validation can reduce conflicting definitions.
6. Impact analysis: changes to shared artifacts can be analyzed for affected components.
7. Documentation: automatically maintained models and reports can reduce manual documentation effort.

Thus, CASE tools can improve productivity, consistency and traceability across the SDLC when used with sound development practices.

14. Explain the constructs used in Structured English (sequence, selection, iteration). Transform a complex multi-condition business rule into Structured English. [7 Marks]

Structured English uses controlled English and programming-like constructs.

1. Sequence: statements execute in order.
2. Selection: IF/THEN/ELSE expresses alternative conditions.
3. Iteration: WHILE/REPEAT/DO constructs represent repetition.

Example business rule:

"If a customer is a member and purchases more than \$500, give 15%; if the customer is not a member and purchases more than \$500, give 10%; if the customer is a member and purchases \$500 or less, give 5%; otherwise give no discount."

Structured English:

Structured English

```

IF PurchaseAmount > 500 THEN
    IF Member = YES THEN
        Discount = 15%
    ELSE
        Discount = 10%
    ENDIF
ELSE
    IF Member = YES THEN
        Discount = 5%
    ELSE
        Discount = 0%
    ENDIF
ENDIF

```

15. Discuss the steps involved in performing a formal Economic Feasibility analysis. Explain Net Present Value (NPV) and Break-Even Analysis. [7 Marks]

Economic feasibility determines whether a proposed system is financially justified.

Steps:

1. Identify all relevant initial, recurring and indirect costs.
2. Identify tangible and, where possible, quantified benefits.
3. Establish the time period for analysis.
4. Estimate annual cash flows.
5. Consider the time value of money.
6. Calculate measures such as payback, ROI and NPV.
7. Determine break-even point.
8. Compare alternatives and document assumptions/sensitivity to uncertainty.

NPV discounts future cash flows to their present value and subtracts the initial investment:

$$NPV = \sum [\text{Cash Flow}_t / (1+r)^t] - \text{Initial Investment}$$

where r is the discount rate and t is the period.

A positive NPV indicates that, under the selected assumptions and discount rate, the discounted benefits/cash flows exceed the initial investment.

Break-even Analysis identifies the point at which cumulative benefits/revenues equal cumulative costs. After break-even, the project begins generating net positive financial value under the chosen model.

16. Explain Software Quality Assurance (SQA). Compare formal Code Inspections, Structured Walkthroughs, and Technical Reviews in defect identification. [7 Marks]

Software Quality Assurance is the planned set of processes, standards, reviews, testing practices and controls used to ensure that software development and maintenance follow defined quality requirements.

Code Inspection:

- Highly formal and systematic.
- Uses preparation, defined roles, checklists and defect logging.
- Focuses closely on finding defects in code or other work products.

Structured Walkthrough:

- Author/team presents the artifact step-by-step to peers.
- Encourages questions and discussion.
- Less formal than an inspection but useful for understanding and defect discovery.

Formal Technical Review:

- Broader technical evaluation of requirements, architecture, design, code, test plans or other artifacts.
- Checks correctness, consistency, standards and fitness for purpose.

All three can detect defects before or alongside testing; their exact formality and scope differ.

SECTION C — LONG ANSWERS

17. COMPULSORY CASE STUDY — AUTOMATED PUBLIC TRANSIT & SMART TICKETING [15 Marks]

The STATS scenario includes RFID smart cards, mobile-banking top-ups, tap-in/tap-out terminals, distance-based fares, driver GPS terminals, dispatch monitoring, rerouting, passenger metrics and fare-revenue auditing. □filecite□turn0file0□L336-L362□

a. Stakeholders and requirements

Stakeholder	Interaction
Commuters	Purchase/register RFID cards, top up balance, tap in/out, receive fare/balance information.
Drivers	Log into GPS terminals and transmit live route coordinates; receive relevant route/dispatch instructions.
Dispatchers/Fleet controllers	Monitor vehicle locations/schedules and issue reroute dispatches.
Banking Gateway	Processes mobile-banking top-ups and returns payment status.
Transit management	Uses passenger traffic metrics and fare revenue audit reports.
Operations/maintenance staff	Support RFID gates, terminals, networks and operational infrastructure.

Three functional requirements:

- The system shall support RFID card registration/purchase and balance management.
- The system shall process tap-in/tap-out events and automatically calculate/deduct distance-based fares.
- The system shall receive GPS data, support dispatch monitoring/rerouting, and produce passenger/fare reports.

Two non-functional requirements:

- Security: payment, card and operational data must be protected through authentication, authorization and secure communications.
- Availability/performance: fare deduction and gate transactions must respond reliably and quickly enough for passenger flow, while central monitoring remains available.

b. Context Diagram

STATS Level 0 Context Diagram

```
Commuter --> card purchase/top-up/tap events -->
    +-----+
    | STATS Smart Transit & |
    | Automated Ticketing   |
    +-----+
Commuter <-- fare/balance/transaction status --
Driver --> GPS/login/route data -----> STATS
Driver <-- dispatch/reroute instructions ----- STATS
Dispatcher --> monitoring/reroute commands --> STATS
Dispatcher <-- vehicle/route status ----- STATS
Banking Gateway <---- top-up transaction -----> STATS
Management <----- traffic/revenue reports STATS
```

c. Use Case Diagram

Use Case — Primary interactions

```

Commuter --> (Purchase/Register Smart Card)
Commuter --> (Top Up Balance) -- «include» --> (Process Payment)
Commuter --> (Tap In)
Commuter --> (Tap Out) -- «include» --> (Calculate Fare)
Driver --> (Login to GPS Terminal)
Driver --> (Transmit GPS Coordinates)
Dispatcher --> (Monitor Fleet)
Dispatcher --> (Issue Reroute)
Management --> (View Traffic Metrics)
Management --> (Audit Fare Revenue)
Banking Gateway --> (Process Payment)

```

«include» = required sub-function reused by a use case.

«extend» = optional/conditional behavior added to a base use case.

d. Installation strategy for minimum passenger disruption

A staged rollout is appropriate as a general migration pattern for a public-facing ticketing system because it can limit the operational exposure of each change. A practical plan is:

23. Pilot the RFID system on a limited route/station or selected gates while paper tickets remain available.
24. Run controlled overlap/parallel operation during validation so transactions and fare calculations can be reconciled.
25. Train drivers, station/operations staff and support personnel before each expansion.
26. Expand route-by-route or station-by-station after meeting reliability, transaction accuracy and support-readiness criteria.
27. Maintain fallback procedures for failed cards, gates, network outages and payment failures.
28. Monitor transaction queues, passenger throughput, balance accuracy and incident rates during each phase.
29. Retire paper tickets only after the new process is stable and contingency arrangements are proven.

18. Critically evaluate the Agile Development Methodology (Scrum/XP) versus the Spiral Model. Contrast their sprint/iterative delivery, client involvement, risk handling, and artifact documentation. Discuss when an organization should choose Agile over structured SDLC. [15 Marks]

Aspect	Agile (Scrum/XP)	Spiral
Delivery	Short iterations/sprints produce frequent increments.	Repeated cycles combine development with explicit risk analysis.
Client involvement	Frequent feedback and product-owner/customer involvement.	Stakeholders participate in defining/evaluating objectives and risk decisions each cycle.
Risk handling	Risks are addressed through short feedback loops, testing, integration and continuous reprioritization.	Risk identification and mitigation are a central explicit feature of each spiral cycle.
Documentation	Focuses on useful documentation; exact level depends on project and governance needs.	Can support substantial documentation of objectives, risks, alternatives and decisions.
Change	Designed to accommodate changing priorities and requirements.	Supports change between cycles, especially when risks/requirements are uncertain.
Planning	Detailed planning is often near-term with a prioritized backlog.	Cycle planning includes risk assessment and objectives before proceeding.

An organization should consider Agile when requirements are expected to evolve, stakeholders can provide frequent feedback, small increments can be delivered safely, and the organization has a team capable of iterative planning, testing and continuous integration. A more structured SDLC may be needed where requirements are tightly controlled, extensive formal documentation/approvals are mandatory, or

governance and contractual constraints demand sequential gates. The decision should be based on project context rather than assuming Agile is universally suitable.

19. Discuss Object Modeling with UML in detail. Explain Sequence Diagrams and Collaboration (Communication) Diagrams. Draw a Sequence Diagram showing object interactions (Lifelines, Messages, Activation Bars) for a User Authentication and Password Reset workflow. [15 Marks]

Object modeling with UML represents a system through objects/classes, their attributes, operations, relationships and interactions. Structural diagrams describe organization; behavioral/interaction diagrams describe dynamic behavior.

Sequence Diagram: emphasizes the chronological order of messages between objects. Lifelines show participating objects over time; activation bars show periods during which an object is executing; messages show calls/signals; returns show responses.

Communication (Collaboration) Diagram: emphasizes the links among participating objects and the messages exchanged over those links. Messages are commonly numbered to indicate sequence.

Sequence Diagram — Authentication and Password Reset



Activation bars should be drawn as vertical rectangles on the lifelines of Login UI, AuthService, UserDB and EmailService during their active processing periods. The important exam elements are the lifelines, ordered messages, activation periods, and the separation of authentication and password-reset interactions.

20. Explain the principles of User Interface (UI) and Output Design. Discuss human factors in UI engineering, screen layout structuring, data validation controls, and error alerting strategies. Explain the importance of output forms and report generation for management decision-making. [15 Marks]

UI design determines how users interact with a system; output design determines how processed information is presented to users and decision-makers. Both should make information accurate, understandable, timely and useful.

Human factors:

- Users have different experience levels, attention, memory and accessibility needs.

- Use familiar terminology and predictable behavior.
- Reduce unnecessary cognitive load and repetitive input.
- Provide clear feedback and visible system status.
- Maintain consistency in navigation, controls, labels and terminology.
- Support accessibility, readable text, logical keyboard navigation and appropriate error recovery.

Screen layout structuring:

- Group related information together.
- Use a clear visual hierarchy: title → sections → fields → actions.
- Keep important information prominent.
- Use consistent alignment, spacing and control placement.
- Provide clear navigation and distinguish primary/secondary actions.
- Avoid clutter and unnecessary decoration that competes with task information.

Data validation controls:

Control	Purpose
Required-field check	Prevents omission of mandatory information.
Data type/format validation	Ensures correct type, date, email, number or code format.
Range validation	Ensures a value lies within acceptable minimum/maximum limits.
Lookup/drop-down controls	Restrict entries to approved values.
Cross-field validation	Checks logical relationships between fields.
Database constraints	Provide server/database-level integrity beyond the interface.

Error alerting strategies:

- Identify the exact problem rather than displaying a vague error.
- Explain how the user can correct it.
- Place the message near the relevant field when practical.
- Avoid technical jargon when communicating with non-technical users.
- Do not erase valid user input after an error.
- Use confirmation for irreversible/high-impact actions.
- Record technical details in logs while presenting an appropriate user-facing message.

Output forms and reports:

Operational outputs such as receipts, confirmations and transaction statements support immediate activities. Management reports aggregate and summarize information for monitoring and decision-making. Good reports should have clear titles, reporting periods, units, meaningful grouping/sorting, accurate calculations, exception indicators, appropriate detail level, and traceable source data. Effective output design turns system data into information that managers can interpret and act upon.

LAST-NIGHT REVISION SHEET

High-frequency definitions and contrasts appearing repeatedly across the four supplied model sets.

Topic	Remember this core point
DFD	Shows data movement between external entities, processes and data stores.
Context Diagram / Level 0	Whole system as one process + external entities + major data flows.
DFD balancing	Parent inputs/outputs remain represented in child decomposition.
Black Hole	Input but no output.
Miracle	Output but no input.
Gray Hole	Outputs not logically supported by shown inputs.
ERD	Entities + attributes + relationships + keys + cardinalities.
Data Dictionary	Central metadata repository defining data elements, structures, stores and flows.
Decision Table	Exhaustive condition/action combinations.
Decision Tree	Branching decision paths.
Structured English	Procedural logic using sequence, selection and iteration.
Functional requirement	What the system does.
Non-functional requirement	Quality/constraint: performance, security, usability, etc.
Technical feasibility	Can we build/support it with available technology and skills?
Operational feasibility	Will it work in the organization and be accepted/used?
Economic feasibility	Do benefits justify costs?
Schedule feasibility	Can it be delivered in the required time?
Logical design	Technology-independent logical structure/behavior.
Physical design	Actual implementation technology and physical structures.
Cohesion	Strength of relationship within a module; high functional cohesion is desirable.
Coupling	Dependency between modules; loose/data coupling is desirable.
Unit testing	Individual component.
Integration testing	Interactions/interfaces among components.
System testing	Complete integrated system.
UAT	User/business acceptance.
Corrective maintenance	Fix faults.
Adaptive maintenance	Adapt to environment changes.
Perfective maintenance	Improve/add/refine functionality or quality.
Preventive maintenance	Reduce future failure/maintenance risk.
1NF	Atomic values; no repeating groups.
2NF	1NF + no partial dependency on composite key.
3NF	2NF + no transitive dependency.
Waterfall	Sequential; lower requirement flexibility.
Prototyping	Early model for feedback/requirement clarification.
Spiral	Risk-driven iterative cycles.
Incremental	System delivered in usable increments.
Agile	Short iterative delivery + frequent feedback.
Use Case	Actor goals/functions.
Class Diagram	Static structure.
Sequence Diagram	Time-ordered interaction.
State Diagram	States and event-driven transitions.

Exam technique: For 2 marks, give definition + one key point/example. For 7 marks, use a definition/introduction followed by 5–7 clearly labeled points or a comparison table. For 15 marks, use an introduction, organized subsections, diagram/table where requested, and a short concluding statement. The supplied paper explicitly allocates 2, 7 and 15 marks to the three sections. □filecite□turn0file0□L28-L60□